

【项目】 抖音电商平台新人礼券使用率优化策略 - 钱鸿林

一、业务背景

抖音电商为吸引新用户下单，会给未在商城下单的新用户发放新人礼券，新用户进入商城页面时，会弹出优惠券的页面，新用户自动领取到优惠券，优惠券有以下两种类型：

- 1) 无门槛优惠券（10元）
- 2) 折扣券（6折最高减10元）

因为营销预算受限，两类优惠券会随机发放给新用户，无门槛优惠券给30%的用户发放，折扣券给70%的用户发放，优惠券的有效期为24小时，未使用的优惠券到期后会自动失效，无法再次领取。

现在营销预算进一步缩减，运营人员考虑调整投放策略。

数据

User_id: 用户ID

IP_city: IP所在城市

Phone_os: 手机系统

Register_date: 注册日期

Is_live_tip: 是否直播打赏，领券前是否在直播间打赏，有打赏记录记为1，无记为0

Live_hour: 浏览直播时长，领券前浏览直播的时长

Video_hour: 浏览短视频时长，领券前浏览短视频的时长

Coupon_date: 领券日期

Coupon_type: 券类型

Coupon_detail: 券明细

Order_date: 用券日期

Order_amount: 用券订单金额

Order_discount_amount: 优惠金额，使用优惠券的折扣金额



coupon_data.csv



54.74MB

SQL题（选做）

- 1、计算每月的领取新人礼券的人数、使用新人礼券的人数、领券到使用的转化率。维度需包含汇总、不同优惠券类型，需一次性输出。
- 2、输出2024-07-22 ~ 2024-07-30日领券的用户中，用券订单金额最小的用户，如存在用户订单金额相同，取user_id最小的记录，输出领券日期和领券后的下单金额最小的用户ID。

数据分析师需分析以下问题：

1) 折扣券的人均营销成本是否低于无门槛优惠券？

折扣券的人均营销成本是否低于无门槛优惠券？

1. 无门槛优惠券（10元）

- 发券成本固定：
 $Cost_{\text{无门槛}} = 10 \text{ 元}$

2. 6折封顶10元券

- 对第 i 位领券用户，设其下单金额为 $order_amount_i$ ：

$$Discount_i = \begin{cases} \min(order_amount_i \times (1 - 0.6), 10), & order_amount_i > 0, \\ 0, & order_amount_i = 0 \end{cases}$$

- 实际使用优惠券 N 位领券用户的人均营销成本：

$$Cost_{\text{6折封顶}} = \frac{1}{N} \sum_{i=1}^N Discount_i$$

3. 成本对比判断

- 若

$$Cost_{\text{6折封顶}} < Cost_{\text{无门槛}} = 10 \text{ 元}$$

则说明 6折封顶10元券 的人均营销成本更低；

- 否则，则 无门槛优惠券 的人均营销成本更低或相等。

Code block

```
1 # 定义计算每位用户实际营销成本的函数
2 def calc_cost(row):
3     if row['Coupon_type'] == '无门槛优惠券':
4         return 10.0
5     elif row['Coupon_type'] == '折扣券':
6         amt = row.get('Order_amount', 0) or 0
7         return min(amt * (1 - 0.6), 10.0) if amt > 0 else 0.0
8
9 # 计算成本列
```

```

10 df['cost'] = df.apply(calc_cost, axis=1)
11
12 # 统计各类型券的人均成本
13
14 # 先过滤：只保留已使用（有下单日期）的优惠券
15 used_df = df[pd.notnull(df['Order_date'])]
16
17 avg_cost = (
18     used_df.groupby('Coupon_type')['cost']
19     .mean()
20     .reset_index()
21     .rename(columns={'cost': 'avg_cost'})
22 )
23
24 avg_cost
25

```

结论：折扣券人均成本 6.91 元，低于无门槛券 10.00 元。

假设检验验证显著性：

Code block

```

1  from statsmodels.stats.weightstats import ztest
2
3  # 使用优惠券的人 cost 列
4  disc = used_df.loc[used_df['Coupon_type'] == '折扣券',
5  'cost'].dropna().to_numpy()
6  nothr = used_df.loc[used_df['Coupon_type'] == '无门槛优惠券',
7  'cost'].dropna().to_numpy()
8
9  # 双尾 z 检验
10 # H0: 两组均值相等  $\mu_{disc} = \mu_{nothr}$ 
11 # H1: 两组均值不相等  $\mu_{disc} \neq \mu_{nothr}$ 
12 z_twoside, p_twoside = ztest(
13     disc,          # 样本1: 折扣券用户成本
14     nothr,         # 样本2: 无门槛券用户成本
15     value=0,       # 假设的均值差（这里是0，表示假设两组均值相等）
16     alternative='two-sided' # 双尾检验，检验“是否有差异”，不关心方向
17 )
18
19 print(f"\n双尾 z 检验: z={z_twoside}, p={p_twoside}")
20
21 # 单尾 z 检验
22 # H0: 折扣券成本  $\geq$  无门槛券成本  $\mu_{disc} \geq \mu_{nothr}$ 

```

```

21 # H1: 折扣券成本 < 无门槛券成本  $\mu_{disc} < \mu_{nothr}$ 
22 z_left, p_left = ztest(
23     disc,          # 样本1: 折扣券用户成本
24     nothr,         # 样本2: 无门槛券用户成本
25     value=0,       # 假设的均值差 (0 表示假设两组均值相等)
26     alternative='smaller' # 单尾检验 (左尾), 检验“第一组 < 第二组”
27 )
28
29 print(f"单尾 z 检验(折扣<无门槛): z={z_left}, p={p_left}")
30
31 alpha = 0.05
32
33 # 双尾结论
34 if p_twoside < alpha:
35     print(f"\n结论1【双尾】: 在显著性水平 {alpha} 下, 两种券的人均成本存在显著差异 (p={p_twoside}) 。")
36 else:
37     print(f"\n结论1【双尾】: 在显著性水平 {alpha} 下, 未发现两种券的人均成本存在显著差异 (p={p_twoside}) 。")
38
39 # 单尾结论 (“折扣券 < 无门槛券”)
40 if p_left < alpha:
41     print(f"结论2【单尾】: 折扣券人均成本显著低于无门槛券 (p={p_left}) 。")
42 else:
43     print(f"结论2【单尾】: 折扣券人均成本不显著低于无门槛券 (p={p_left}) 。")
44

```

结论:

双尾 z 检验: $z=-155.10652365387426$, $p=0.0$

单尾 z 检验(折扣<无门槛): $z=-155.10652365387426$, $p=0.0$

结论1【双尾】: 在显著性水平 0.05 下, 两种券的人均成本存在显著差异。

结论2【单尾】: 折扣券人均成本显著低于无门槛券。

2) 如果折扣券的人均营销成本较低, 是否存在一种类型的用户, 领券折扣券下单的概率大于或等于领券无门槛优惠券后下单的概率?

方法: 因果推断

(为什么这里不能用DID 和 PSM:

- 1、DID是需要有时间趋势，比如说领折扣券前后商家销售额，用户购买量改变了多少这种，才能用
- 2、PSM来说这不是“总体上折扣券效果是不是比无门槛券好”，而是“对于某些特定用户，折扣券效果 $\geq$ 无门槛券”。PSM 估的是“整体匹配后平均”效果，回答不了“哪类用户的个体化增益 $\Delta(X)$ 比较”)

步骤一：生成模型需要的变量和特征工程

1. 生成干预标签 (treatment)

- 根据 `Coupon_type` 映射两种策略:
 - “无门槛优惠券” $\rightarrow$ `no_threshold`
 - “折扣券” $\rightarrow$ `discount`

2. 构造目标变量 (y)

- 新增 `coupon_used_flag`:
 - 若 `Order_date` 不为空，则标记为 1（使用优惠券）
 - 否则为 0

3. 定义特征集

- 连续特征
 - `register_to_coupon_days`、`days_to_order`
 - `Live_hour`、`Video_hour`
 - `Order_amount`、`Order_discount_amount`
- 类别特征
 - `Is_live_tip`、`IP_city`、`Phone_os`

Code block

```
1 csv_path = '/Users/qianhonglin/Desktop/Data Analyst/Project 1 抖音电商平台新人礼券
  使用率优化策略/data/coupon_data.csv'
2 Df = pd.read_csv(csv_path)
3
4 df = Df.copy()
5 for col in ['Register_date', 'Coupon_date', 'Order_date']:
6     df[col] = pd.to_datetime(df[col], format='%Y/%m/%d', errors='coerce')
7
8 # 生成 treatment (用作某种干预t)
```

```

9 df['treatment'] = df['Coupon_type'].map({
10     '无门槛优惠券': 'no_threshold',
11     '折扣券': 'discount'
12 })
13
14 # 生成优惠券标识 (预测变量y)
15 df['coupon_used_flag'] = df['Order_date'].notna().astype(int) # 是否使用了优惠券
    (0/1)
16
17 # print(df.columns)
18 # Index(['User_id', 'IP_city', 'Phone_os', 'Register_date', 'Is_live_tip',
19 #        'Live_hour', 'Video_hour', 'Coupon_date', 'Coupon_type',
20 #        'Coupon_detail', 'Order_date', 'Order_amount',
21 #        'Order_discount_amount',
22 #        'treatment'],
23 #       dtype='object')
24 # 特征工程
25 df['register_to_coupon_days'] = (df['Coupon_date'] -
    df['Register_date']).dt.days.fillna(0) # 注册到领券的天数差
26 df['days_to_order'] = (df['Order_date'] -
    df['Coupon_date']).dt.days.fillna(0) # 领券到下单的天数
27 df['Live_hour'] = df['Live_hour'].fillna(0) # 直播观看时长 (填 0 表示
    未看)
28 df['Video_hour'] = df['Video_hour'].fillna(0) # 短视频浏览时长 (填 0 表
    示未看)
29 df['Is_live_tip'] = df['Is_live_tip'].fillna(0) # 直播打赏标志 (填
    0/1)
30 df['Order_amount'] = df['Order_amount'].fillna(0) # 下单金额 (填 0 表示
    未下单)
31 df['Order_discount_amount'] = df['Order_discount_amount'].fillna(0) # 优惠金额
    (填 0 表示未使用优惠券)
32
33 # 预处理与划分
34 continuous_features = [
35     'register_to_coupon_days',
36     'days_to_order',
37     'Live_hour',
38     'Video_hour',
39     'Order_amount',
40     'Order_discount_amount',
41 ]
42 categorical_features = [
43     'Is_live_tip',
44     'IP_city',
45     'Phone_os'
46 ]

```

```

47     preprocessor = ColumnTransformer(transformers=[
48         ('num', StandardScaler(), continuous_features),
49         ('cat', OneHotEncoder(drop='first', sparse_output=False),
           categorical_features)
50     ])
51

```

步骤二：用uplift模型进行训练和预测

关键变量解释

- **X**
 - 特征矩阵：包含对原始 DataFrame 中所有连续特征做标准化、类别特征做独热编码后的数值化特征，作为模型输入。
- **treatment**
 - 干预组标签数组：每个元素是该样本对应的优惠券策略（'no_threshold' 或 'discount'），用于告诉 uplift 模型哪部分样本属于“对照组”或“实验组”。
- **y**
 - 目标变量数组：由 0/1 组成，表示该样本是否真正使用了优惠券（1 = “最高10元6折券”，0 = “无门槛券”），用于模型拟合和评估。

训练并预测并计算 uplift

- 对测试集调用
`proba = ct.predict(X_test)`

得到两列概率：

 - `proba[:, 0]` = $P(\text{order}=1 \mid \text{no_threshold})$
 - `proba[:, 1]` = $P(\text{order}=1 \mid \text{discount})$
- 计算增量：
`uplift_vals = proba[:, 1] - proba[:, 0]`

Code block

```

1
2     X = preprocessor.fit_transform(df)
3     treatment = df['treatment'].values
4     y = df['coupon_used_flag'].values
5
6     X_train, X_test, t_train, t_test, y_train, y_test = train_test_split(X,
7                                     treatment, y, test_size=0.2, random_state=42)
8
9     # 训练与预测
10    ct = UpliftTreeClassifier(
11        control_name='no_threshold', # 指定对照组标签
12        max_depth=5,                 # 限制树的最大深度为5
13        min_samples_leaf=50          # 每个叶节点上至少要有50个样本
14    )
15    ct.fit(X_train, treatment=t_train, y=y_train)

```

```

15
16 #预测下单概率, 计算 uplift
17 proba = ct.predict(X_test)
18
19 # proba[:, 0] = P(order=1 | 无门槛)
20 # proba[:, 1] = P(order=1 | 最大10折扣)
21
22 # uplift = P(折扣券) - P(无门槛券)
23 uplift_vals = proba[:, 1] - proba[:, 0]
24
25 # 构造测试集上的结果 DataFrame
26 result_df = pd.DataFrame({
27     'P_no_threshold': proba[:, 0],      # 用户在“无门槛”组下被预测为下单 (order=1)
    的概率 (对照组预测概率)
28     'P_discount':      proba[:, 1],      # 用户在“折扣”组下被预测为下单 (order=1) 的
    概率 (试验组预测概率)
29     'model_uplift':    uplift_vals,      # 模型预测的 uplift 值, 即 P_discount -
    P_no_threshold, 表示因干预带来的增量概率
30     'treatment':       t_test,          # 测试集中每个样本的实际分组标签
    ('no_threshold' 或 'discount')
31     'actual_outcome':  y_test           # 测试集中每个样本的真实结果
    (coupon_used_flag, 0/1 表示是否使用了券)
32 })
33
34 # 映射 treatment 为 0/1
35 result_df['treatment_map'] = result_df['treatment'].map({'no_threshold': 0,
    'discount': 1}).astype(int)
36
37 print(f"满足 model_uplift ≥ 0 的用户数: {(result_df['model_uplift'] >= 0).sum()}
    / {len(result_df)}")
38

```

结论：满足 $\text{model_uplift} \geq 0$ 的用户数：77618 / 129425

步骤三：用Qini曲线评估模型好坏

Qini 曲线的意义

- 可视化模型增益分布：Qini 曲线将模型预测的个体增量 (uplift) 按照大小排序后，展示在不同分位 (Population Quantile) 上真实累积增益 (Cumulative Uplift) 的变化。
- 评估精准投放效果：通过对比模型曲线与随机基线 (Random Baseline) 间的偏离程度，可以直观判断模型在前几分位上能否更精准地识别出高增益人群。
- 辅助阈值决策：曲线陡峭的区间 (曲线远高于随机线) 对应的是“高效区”，可据此确定营销预算或投放比例的最佳阈值。

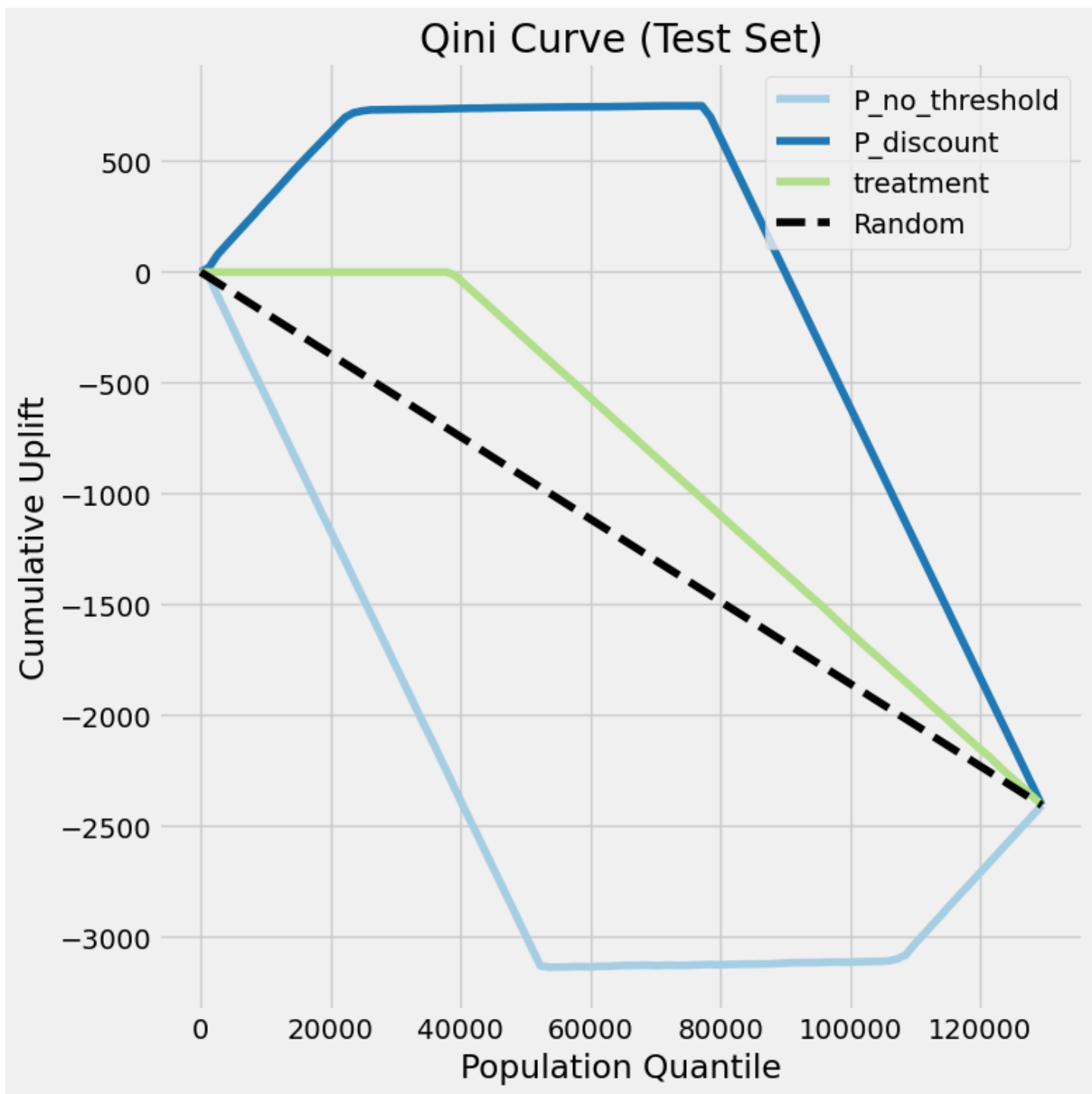
Qini Score 的意义

- 量化模型优劣：Qini Score 是模型曲线与随机基线之间面积差的数值表示，面积越大说明模型在各分位累计增益越显著，效果越好。
- 比较不同模型 / 策略：在多个 uplift 模型或多种处理策略之间，以 Qini Score 高低作为直接对比依据，选出最优方案。

Code block

```
1  # 画 Qini 曲线
2  plt.figure(figsize=(13, 6), dpi=300)
3
4  plot_qini(
5      result_df,                                # DataFrame: 包含实际结果、处理分组和模型
        预测提升值
6      outcome_col='actual_outcome',             # str: 实际观测结果列名, 用于计算累积提升
7      treatment_col='treatment_map',            # str: 处理分组列名, 指示哪些样本接受了处
        理
8      treatment_effect_col='model_uplift',      # str: 模型预测的个体提升效果列名
9      normalize=False                           # bool: 是否对累积提升值进行归一化;
        False 表示使用原始提升值
10 )
11 plt.title("Qini Curve (Test Set)")
12 plt.xlabel("Population Quantile")
13 plt.ylabel("Cumulative Uplift")
14 plt.tight_layout()
15 plt.show()
16
17 # 计算 Qini score
18 score = qini_score(
19     result_df,
20     outcome_col='actual_outcome',
21     treatment_col='treatment_map',
22     treatment_effect_col='model_uplift',
23     normalize=False
24 )
25
26 print(f"模型的 Qini score: {score}")
```

结论:



模型的 Qini score:

P_no_threshold -1219.643509

P_discount 1248.257985

treatment 358.690410

- **P_no_threshold** (浅蓝) 如果你把所有人都当作做了 “no threshold” 处理 (并按模型对这个策略的预测效果给人排序), 那么在每个分位累积的真实增益是多少。
- **P_discount** (深蓝) 如果你把所有人都当作做了 “discount” 处理 (同样按模型预测效果排序), 那么对应的累积增益曲线。
- **treatment** (浅绿) 这是你数据里标记为 `treatment` 的那组人 (在训练好的模型条件下, 按原本的treatment得到的累积增益情况), 按模型预测效果排序后实际观测到的累积增益。
- **Random** (黑虚线) 随机分配给处理的基线: 把人随机分到 “处理” 里, 累积增益应该是一条直线。

核心结论：原本我们的策略是随机投放“无门槛券”和“最高10元6折券”，现在通过uplift模型我们发现：在测试集投放“最高10元6折券优惠券”在前10%-20%的用户增大了累计收益（他们的下单概率变大）

3) 如果存在上述类型的用户，用户特征是怎样的？

步骤一：还原特征

Code block

```
1  import numpy as np
2  import pandas as pd
3  from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
4
5  # 分离数值 & OHE 特征
6  X_scaled = preprocessor.transform(df)
7  n_num = len(continuous_features)
8  X_num_scaled = X_scaled[:, :n_num]
9  X_cat_ohe = X_scaled[:, n_num:]
10
11 # 数值部分逆变换到“原始”尺度
12 # scaler = preprocessor.named_transformers_['num']
13 X_num_orig =
    preprocessor.named_transformers_['num'].inverse_transform(X_num_scaled)
14
15 # 横向拼接：原始数值 + OHE 数值 (0/1)
16 X_rebuilt = np.hstack([X_num_orig, X_cat_ohe])
17
18 # 构造特征名（原数值名 + OHE 名）
19 cat_feature_names =
    preprocessor.named_transformers_['cat'].get_feature_names_out(categorical_features)
20 feature_names = continuous_features + list(cat_feature_names)
21
22 df_repr = pd.DataFrame(X_rebuilt, columns=feature_names)
23
24 df_repr
```

步骤二：找 $\text{model_uplift} \geq 0$ 的用户数

Code block

```

1  # 全数据计算 uplift、生成 mask
2  proba          = ct.predict(X_scaled)                # 或 ct.predict_proba
3  uplift_vals    = proba[:, 1] - proba[:, 0]
4  mask          = uplift_vals >= 0
5
6  result_df = pd.DataFrame({
7      'P_no_threshold': proba[:, 0],
8      'P_discount':     proba[:, 1],
9      'model_uplift':   uplift_vals
10 })
11
12 print(f'满足 model_uplift ≥ 0 的用户数: ' f'{{(result_df["model_uplift"] >=
13      0)}.sum()}} / {{len(result_df)}}')

```

结论：满足 model_uplift ≥ 0 的用户数：388228 / 647122

步骤三：用DecisionTreeClassifier提炼用户特征

Code block

```

1  # 在“原始”特征上重训练决策树
2  rule_tree    = DecisionTreeClassifier(max_depth=7, min_samples_leaf=50)
3  rule_tree.fit(df_repr, mask)
4
5  print(export_text(rule_tree, feature_names=feature_names))
6
7  plt.figure(figsize=(20, 10))
8  plot_tree(
9      rule_tree,
10     feature_names=feature_names,
11     class_names=['uplift < 0', 'uplift ≥ 0'],
12     filled=True,
13     rounded=True
14 )
15
16 plt.tight_layout()
17 plt.show()

```

结论：

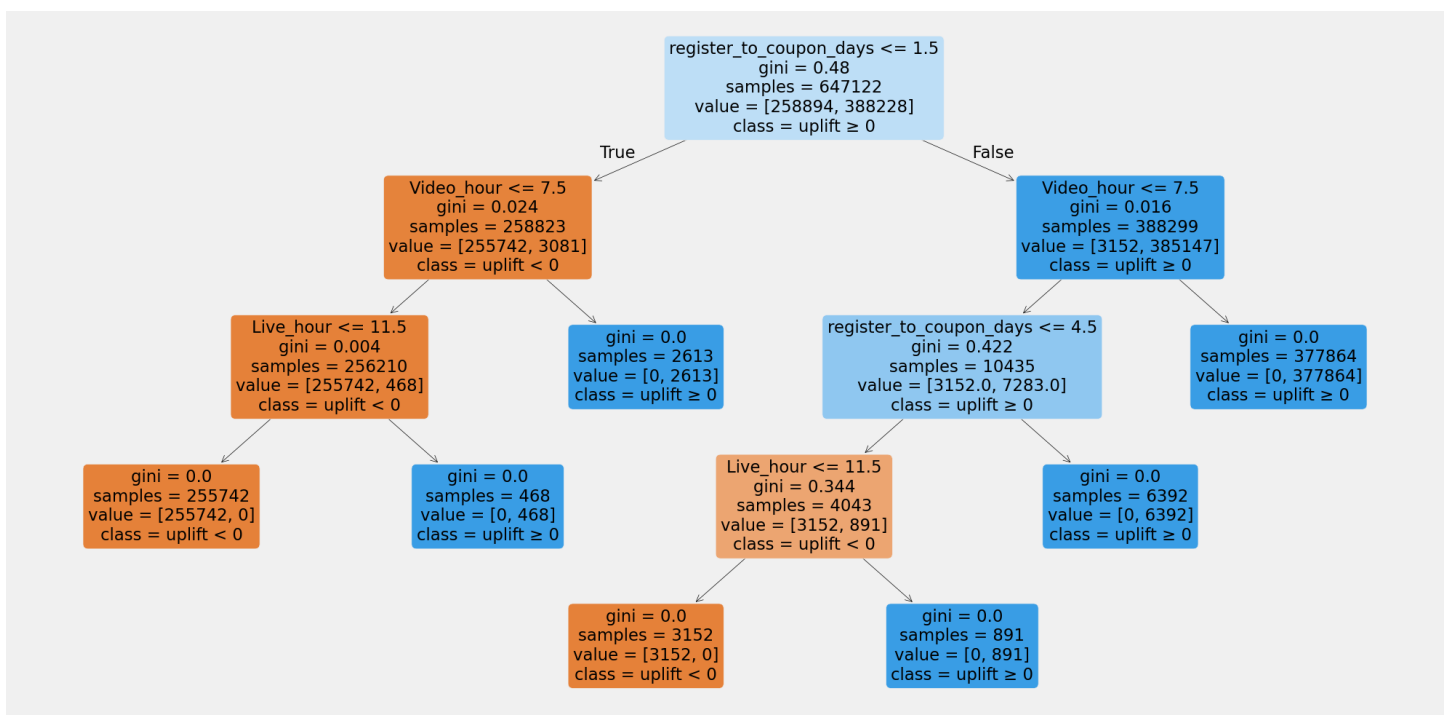
提炼的用户类型规则 (uplift ≥ 0)

|--- register_to_coupon_days ≤ 1.50

```

| |--- Video_hour <= 7.50
| | |--- Live_hour <= 11.50
| | | |--- class: False
| | |--- Live_hour > 11.50
| | | |--- class: True
| |--- Video_hour > 7.50
| | |--- class: True
|--- register_to_coupon_days > 1.50
| |--- Video_hour <= 7.50
| | |--- register_to_coupon_days <= 4.50
| | | |--- Live_hour <= 11.50
| | | | |--- class: False
| | | |--- Live_hour > 11.50
| | | | |--- class: True
| | |--- register_to_coupon_days > 4.50
| | | |--- class: True
| |--- Video_hour > 7.50
| | |--- class: True

```



用户类型结论

1. 快速领券+内容活跃型

- 注册→领券间隔
 - `register_to_coupon_days` ≤ 1.5 说明：注册后非常快就领券。
- 再分两支：
 1. 视频活跃
 - `Video_hour` > 7.5
 - 强烈正响应
 2. 视频不活跃但直播活跃
 - `Video_hour` ≤ 7.5
 - `Live_hour` > 11.5
 - 正响应

2. 慢速领券+视频活跃型

- 注册→领券间隔
 - `register_to_coupon_days` > 1.5 (“较慢”或“中等”节奏)
- 视频活跃
 - `Video_hour` > 7.5
 - 无论直播状况，都是正向增效

3. 慢速领券+“成熟订阅”型

- 注册→领券间隔
 - `register_to_coupon_days` > 1.5
- 视频不活跃
 - `Video_hour` ≤ 7.5
- 从容领券
 - `register_to_coupon_days` > 4.5 (更晚领取)
 - 正响应
 - 说明：不怎么看视频，但慢领券表明他们更有耐心、更价格敏感，6折券能显著驱动下单。

4. 缓冲领券+直播主导型

- 注册→领券间隔
 - `register_to_coupon_days` 在 $[1.5, 4.5]$
- 视频不活跃
 - `Video_hour` ≤ 7.5
- 直播活跃
 - `Live_hour` > 11.5
 - 正响应

- **优先触达**：对视频或直播至少一端较活跃的用户，或者领券节奏偏“从容”而非“一领就走”的用户
- **避免浪费**：视频和直播都不活跃且领券节奏极快的用户

4) 设计AB实验验证这类用户领取折扣券下单的概率大于或等于领去无门槛优惠券后下单的概率？

4.1 实验目的：

验证在已划分为“6折最高10元折扣券正响应”的4组人群中，领取折扣券后 24 小时下单转化率是否显著高于领取无门槛券后的转化率。

4.2 实验策略动作：

如何分组：对目标人群（四个细分 Segment1-4）使用随机数切分，1:1 分配至对照/实验组。

- 对新注册并符合分群条件的用户进行如下操作：
 - **实验组**：发放 6 折（最高减 10 元）折扣券
 - **对照组**：发放 10元无门槛优惠券

核心指标：24 小时内下单转化率（conversion_24h）

分流单位：用户ID

4.3 最小样本量（每个组（实验组和对照组）所需的 最少用户数）：

公式（双侧、两独立样本比例检验）：

$$n_{\text{per group}} = \frac{(Z_{1-\alpha/2} + Z_{1-\beta})^2 [p_1(1 - p_1) + p_2(1 - p_2)]}{(p_1 - p_2)^2}$$

- 其中 $Z_{1-\alpha/2} = 1.96$ （双侧 $\alpha = 0.05$ ）， $Z_{1-\beta} = 0.84$ （ $\beta = 0.2$ ）。
- 需要事先预估：
 - 基线转化率 p_2 （无门槛券时的历史转化率）
 - 最小可检测差异 $\Delta = p_1 - p_2$

4.4 实验假设与检验方法：

p1: 定义为实验组（6折最高10元折扣券）用户的下单转化率

p2: 定义为对照组（无门槛券）用户的下单转化率

- 原假设 (H_0): $p_1 - p_2 = 0$
- 备择假设 (H_1): $p_1 - p_2 \neq 0$ （双侧检验）

设显著性水平 $\alpha = 0.05$ ，检验功效 $1 - \beta = 0.8$

检验方法：

- 双侧两样本比例 z 检验：

$$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1 - \hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}, \quad \hat{p} = \frac{k_1 + k_2}{n_1 + n_2}$$

- 拒绝域： $|z| > z_{1-\alpha/2} = 1.96$ ($\alpha = 0.05$)
- 若 $|z| > 1.96$ ，则拒绝 H_0 ，认为 p_1 与 p_2 存在显著差异。

项目总结

一、背景和结论

1、背景：

抖音电商为吸引新用户下单，会给未在商城下单的新用户发放新人礼券，新用户进入商城页面时，会弹出优惠券的页面，新用户自动领取到优惠券，优惠券有以下两种类型：

- 1) 无门槛优惠券（10元）
- 2) 折扣券（6折最高减10元）

因为营销预算受限，原本两类优惠券会随机发放给新用户，无门槛优惠券给30%的用户发放，折扣券给70%的用户发放，优惠券的有效期为24小时，未使用的优惠券到期后会自动失效，无法再次领取。

现在营销预算进一步缩减，运营人员考虑调整投放策略。

2、实验分析与得到了什么结论，建议是什么

实验主要结论：

- 1、折扣券（6折最高减10元）发放成本整体低于无门槛券
- 2、基于注册到领券间隔和视频/直播活跃度，可将用户一分为四：快速领券且内容活跃型、慢速领券且视频活跃型、慢速领券的“成熟订阅”型，以及领券缓冲期内以直播互动为主的用户。这四类用户领“折扣券（6折最高减10元）”后下单概率大于“无门槛券”

营销建议

- **优先触达**：对视频或直播至少一端较活跃的用户，或者领券节奏偏“从容”而非“一领就走”的用户
- **避免浪费**：视频和直播都不活跃且领券节奏极快的用户

3、建议是否有落地，有的话效果是怎么样

实习结束时验证分析结论的AB实验还在排期，未开始。

二、分析过程和数据表现

1、核心分析思路

- 基于业务方要降低成本的要求调整优惠券发放策略，我们首先要看**领券并且下单的人**里面“折扣券（6折最高减10元）”的成本是不是低于“无门槛优惠券（10元）”
- 确定折扣券的人均营销成本较低，判断是否存在一种类型的用户，领券折扣券**下单的概率大于或等于**领券无门槛优惠券后下单的概率？
- 如果该用户存在，则找出**用户特征**
- 基于每种用户特征**设计ab test**, 验证实验结果的可靠性

2、每一步分析动作

第一步：折扣券的人均营销成本是否低于无门槛优惠券？

1. 无门槛优惠券（10 元）

- 发券成本固定：
 $Cost_{无门槛} = 10 \text{ 元}$

2. 6折封顶10元券

- 对第 i 位领券用户，设其下单金额为 $order_amount_i$ ：
$$Discount_i = \begin{cases} \min(order_amount_i \times (1 - 0.6), 10), & order_amount_i > 0, \\ 0, & order_amount_i = 0 \end{cases}$$
- 实际使用优惠券 N 位领券用户的人均营销成本：
$$Cost_{6折封顶} = \frac{1}{N} \sum_{i=1}^N Discount_i$$

3. 成本对比判断

- 若
$$Cost_{6折封顶} < Cost_{无门槛} = 10 \text{ 元}$$

则说明 6折封顶10元券 的人均营销成本更低；
否则，则 无门槛优惠券 的人均营销成本更低或相等。

结论：折扣券人均成本 6.91 元，低于无门槛券 10.00 元。

再用假设检验验证结果显著性：

双尾 z 检验: z=-155.10652365387426, p=0.0

单尾 z 检验(折扣<无门槛): z=-155.10652365387426, p=0.0

结论1【双尾】：在显著性水平 0.05 下，两种券的人均成本存在显著差异。

结论2【单尾】：折扣券人均成本显著低于无门槛券。

第二步：如果折扣券的人均营销成本较低，是否存在一种类型的用户，领券折扣券下单的概率大于或等于领券无门槛优惠券后下单的概率？

1. 生成需要的变量和特征工程

A. 生成干预标签（treatment）

- 根据原始字段 `Coupon_type` 映射两种策略：
 - “无门槛优惠券” → `no_threshold`
 - “折扣券” → `discount`

B. 构造目标变量（y）

- 新增二元响应变量 `coupon_used_flag`：
 - 若 `Order_date` 不为空，则赋值 1（表示使用了优惠券）
 - 否则赋值 0（未使用优惠券或未下单）

C. 定义特征集

a. 连续型特征

- `register_to_coupon_days`：领券日 - 注册日（天）
- `days_to_order`：下单日 - 领券日（天）
- `Live_hour`：直播观看时长（小时，缺失填 0）
- `Video_hour`：短视频浏览时长（小时，缺失填 0）
- `Order_amount`：下单金额（缺失填 0）
- `Order_discount_amount`：优惠金额（缺失填 0）

b. 类别型特征

- `Is_live_tip`：是否在直播中打赏（0/1，缺失填 0）
- `IP_city`：用户所在城市（类别编码）
- `Phone_os`：手机操作系统（类别编码）

2. 使用 Uplift 模型进行训练与预测

a. 关键变量说明

- **X**：预处理后的特征矩阵，包含对所有连续特征做标准化、类别特征做独热编码后的数值化特征，用于模型输入。
- **treatment**：干预标签数组，每个样本对应的优惠券策略（`'no_threshold'` 或 `'discount'`），用于告诉 Uplift 模型哪些样本属于对照组、哪些属于实验组。
- **y**：目标变量数组，由 0/1 组成，表示该样本是否真正使用了优惠券（1=用券下单，0=未用券或未下单），用于模型拟合和评估。

b. 模型训练

- 根据 `X_train`、`treatment_train`、`y_train` 三部分数据，调用 Uplift 模型的 `fit` 方法进行训练。

Code block

```
1 uplift_model.fit(X_train, treatment_train, y_train)
```

c. 对测试集进行预测

- i. 使用训练好的模型对 `X_test` 调用 `predict_proba` :

Code block

```
1 proba = uplift_model.predict_proba(X_test)
```

- 返回结果是一个形如 `(n_samples, 2)` 的数组：
 - `proba[:, 0]` = $P(y=1 \mid \text{对照组} = \text{no_threshold})$
 - `proba[:, 1]` = $P(y=1 \mid \text{实验组} = \text{discount})$

d. 计算 Uplift 分数

- i. 对每个样本，用实验组响应率减去对照组响应率，即可得到 Uplift 值：

Code block

```
1 uplift_vals = proba[:, 1] - proba[:, 0]
```

- 正值表示该用户在给折扣券时相比无门槛券更容易下单；负值则说明折扣券反而降低了下单概率。

e. 结果关联与保存

- 将 `uplift_vals` 与 `User_id` 或原始样本索引合并，生成每个用户的 Uplift 评分列表，便于后续分层分析和投放决策。

至此，我们完成了 Uplift 模型的训练与预测，并得到了可用于人群分层的增量效果分数。

结论：在测试集上满足 $\text{model_uplift} \geq 0$ 的用户数：77618 / 129425

3. 用 Qini 曲线和 Qini Score 进行模型评价

• 可视化模型增益分布

Qini 曲线将模型预测的个体增量 (uplift) 按照大小排序后，展示在不同分位 (Population Quantile) 上的真实累计增益 (Cumulative Uplift) 。

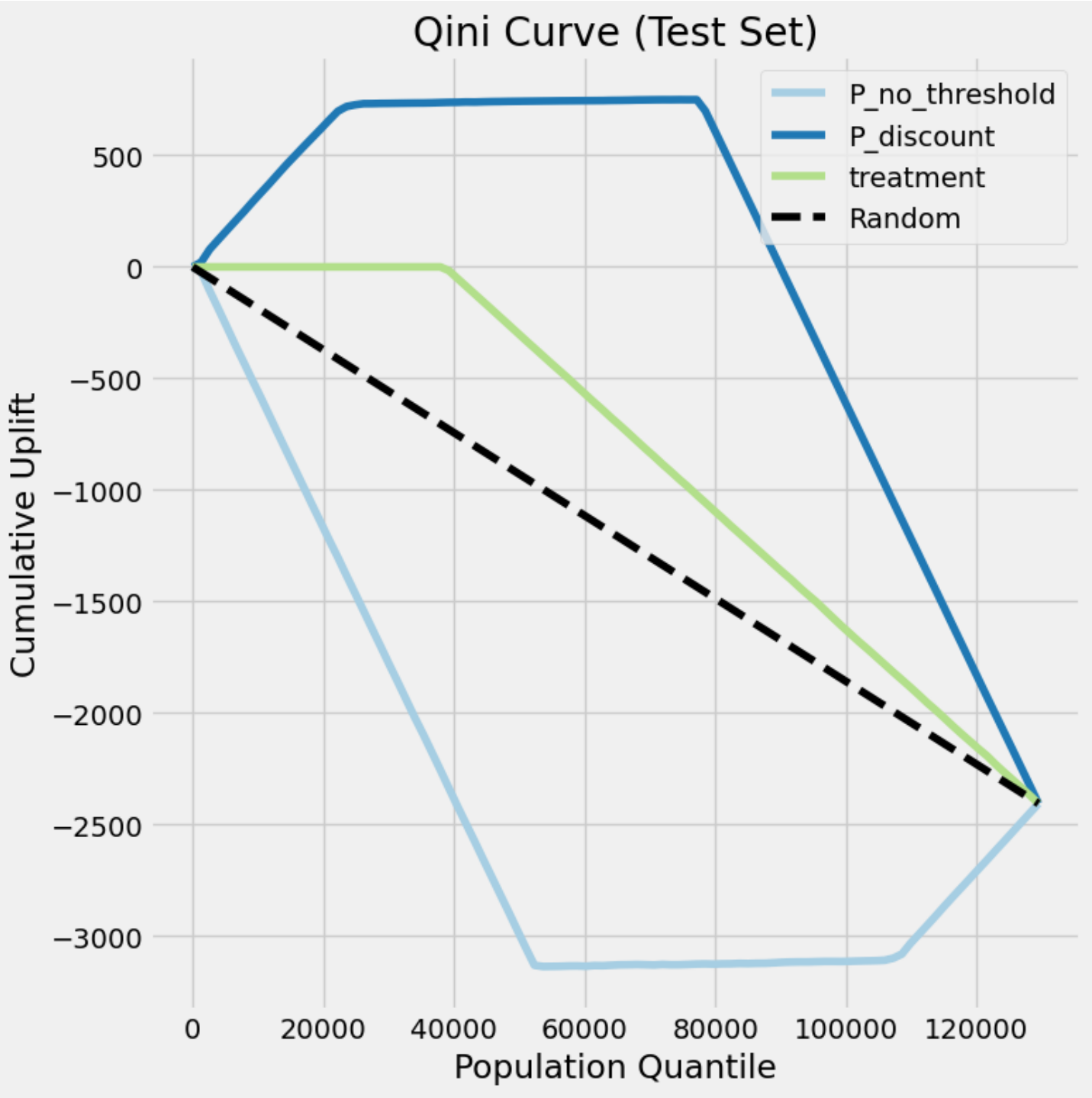
• 评估精准投放效果

通过对比模型曲线与随机基线 (Random Baseline) 之间的偏离程度，可以直观判断模型在前几分位上能否更精准地识别出高增益人群。

• 辅助阈值决策

曲线陡峭的区间（即曲线明显高于随机线的部分）对应“高效区”，可据此确定营销预算或投放比例的最佳阈值。

实验结果：



模型的 Qini score:

P_no_threshold -1219.643509

P_discount 1248.257985

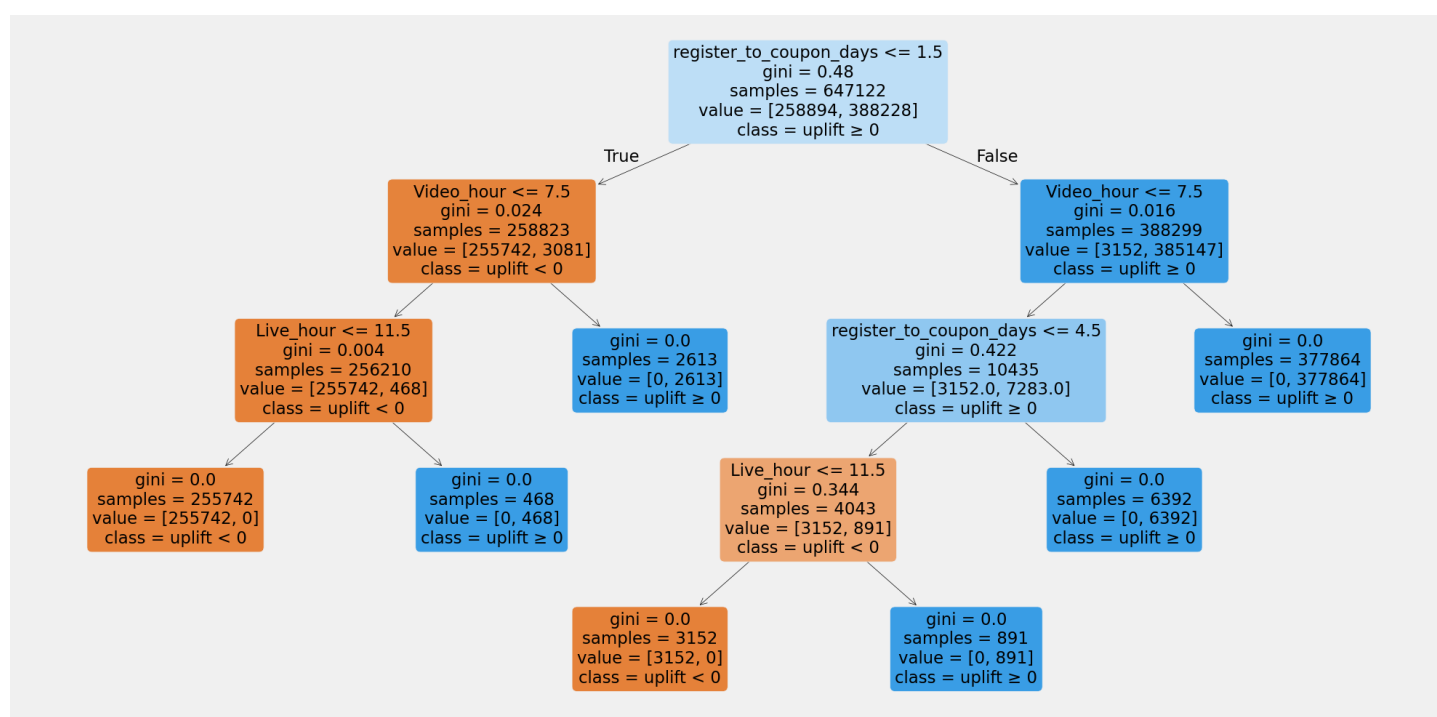
treatment 358.690410

核心结论：原本我们的策略是随机投放“无门槛券”和“最高10元6折券”，现在通过uplift模型我们发现：在测试集投放“最高10元6折券优惠券”在前10%-20%的用户增大了累计收益（他们的下单概率变大）

第三步：如果存在上述类型的用户，用户特征是怎样的？

- 先在全数据集上还原特征
- 然后找 $\text{model_uplift} \geq 0$ 的用户数（满足 $\text{model_uplift} \geq 0$ 的用户数：388228 / 647122）
- 最后用DecisionTreeClassifier提炼用户特征

实验结果：



用户类型结论

a. 快速领券+内容活跃型

- 注册→领券间隔： $\text{register_to_coupon_days} \leq 1.5$ （注册后非常快就领券）
- 再分两支：
 - 视频活跃： $\text{Video_hour} > 7.5$ → 强烈正响应
 - 视频不活跃但直播活跃： $\text{Video_hour} \leq 7.5$ 且 $\text{Live_hour} > 11.5$ → 正响应

b. 慢速领券+视频活跃型

- 注册→领券间隔： $\text{register_to_coupon_days} > 1.5$ （“较慢”或“中等”节奏）
- 视频活跃： $\text{Video_hour} > 7.5$ ，无论直播状态，都能获得正向增效

c. 慢速领券+“成熟订阅”型

- 注册→领券间隔: `register_to_coupon_days > 1.5`
- 视频不活跃: `Video_hour ≤ 7.5`
- 从容领券: `register_to_coupon_days > 4.5` (更晚领取) → 正响应
- 说明: 不怎么看视频, 但慢领券表明更有耐心、更价格敏感, 6折券能显著驱动下单

d. 缓冲领券+直播主导型

- 注册→领券间隔: `1.5 < register_to_coupon_days ≤ 4.5`
- 视频不活跃: `Video_hour ≤ 7.5`
- 直播活跃: `Live_hour > 11.5` → 正响应

第四步: 设计AB实验验证这类用户领取折扣券下单的概率大于或等于领取无门槛优惠券后下单的概率?

1. 实验目的:

验证在已划分为“6折最高10元折扣券正响应”的4组人群中, 领取折扣券后 24 小时下单转化率是否显著高于领取无门槛券后的转化率。

2. 实验策略动作:

如何分组: 对目标人群 (四个细分 Segment1-4) 使用随机数切分, 1:1 分配至对照/实验组。

- 对新注册并符合分群条件的用户进行如下操作:

- **实验组:** 发放 6 折 (最高减 10 元) 折扣券
- **对照组:** 发放 10元无门槛优惠券

核心指标: 24 小时内下单转化率 (conversion_24h)

分流单位: 用户ID

3. 最小样本量 (每个实验组和对照组所需的 最少用户数):

公式（双侧、两独立样本比例检验）：

$$n_{\text{per group}} = \frac{(Z_{1-\alpha/2} + Z_{1-\beta})^2 [p_1(1-p_1) + p_2(1-p_2)]}{(p_1 - p_2)^2}$$

- 其中 $Z_{1-\alpha/2} = 1.96$ （双侧 $\alpha = 0.05$ ）， $Z_{1-\beta} = 0.84$ （ $\beta = 0.2$ ）。
- 需要事先预估：
 - 基线转化率 p_2 （无门槛券时的历史转化率）
 - 最小可检测差异 $\Delta = p_1 - p_2$

4. 实验假设与检验方法：

p1: 定义为实验组（6折最高10元折扣券）用户的下单转化率

p2: 定义为对照组（无门槛券）用户的下单转化率

- 原假设 (H_0): $p_1 - p_2 = 0$
- 备择假设 (H_1): $p_1 - p_2 \neq 0$ （双侧检验）

设显著性水平 $\alpha = 0.05$ ，检验功效 $1 - \beta = 0.8$

检验方法：

- 双侧两样本比例 z 检验：

$$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}, \quad \hat{p} = \frac{k_1 + k_2}{n_1 + n_2}$$

- 拒绝域： $|z| > z_{1-\alpha/2} = 1.96$ （ $\alpha = 0.05$ ）
- 若 $|z| > 1.96$ ，则拒绝 H_0 ，认为 p_1 与 p_2 存在显著差异。

三、复盘空间

1、之前做的部分有哪些是可以优化的

1. 数据与特征

- 引入更多用户行为特征：历史消费频次/金额、活跃天数等。

2. 模型与评估

- 比较多种 Uplift 算法：Uplift Random Forest、Uplift XGBoost 等
- 考虑因果树，因果森林等因果推断的异质性分析模型。
- 参数优化：网格搜索，optuna参数优化

3. ab实验设计

- 优化对照组/实验组样本比例，做分层抽样，确保各细分人群均衡覆盖
- 在不同地域或渠道上做分批验证，测试模型的泛化能力。

2、未来还可以做些什么，来进一步解决业务问题

基于业务问题是优惠券营销成本高需要优化，实施优惠券策略优化

- 面额与类别调整：
 - 降低成本目标：降低无门槛优惠券面额（如由 10 元降至 5 元），测试最低有效阈值。
 - 提高优惠券转化率目标：延长优惠券的有效期，例如24小时延长到48小时
- 有效期优化：缩短/延长券有效期，寻找最佳时效窗口，提升使用率同时控制损耗。

